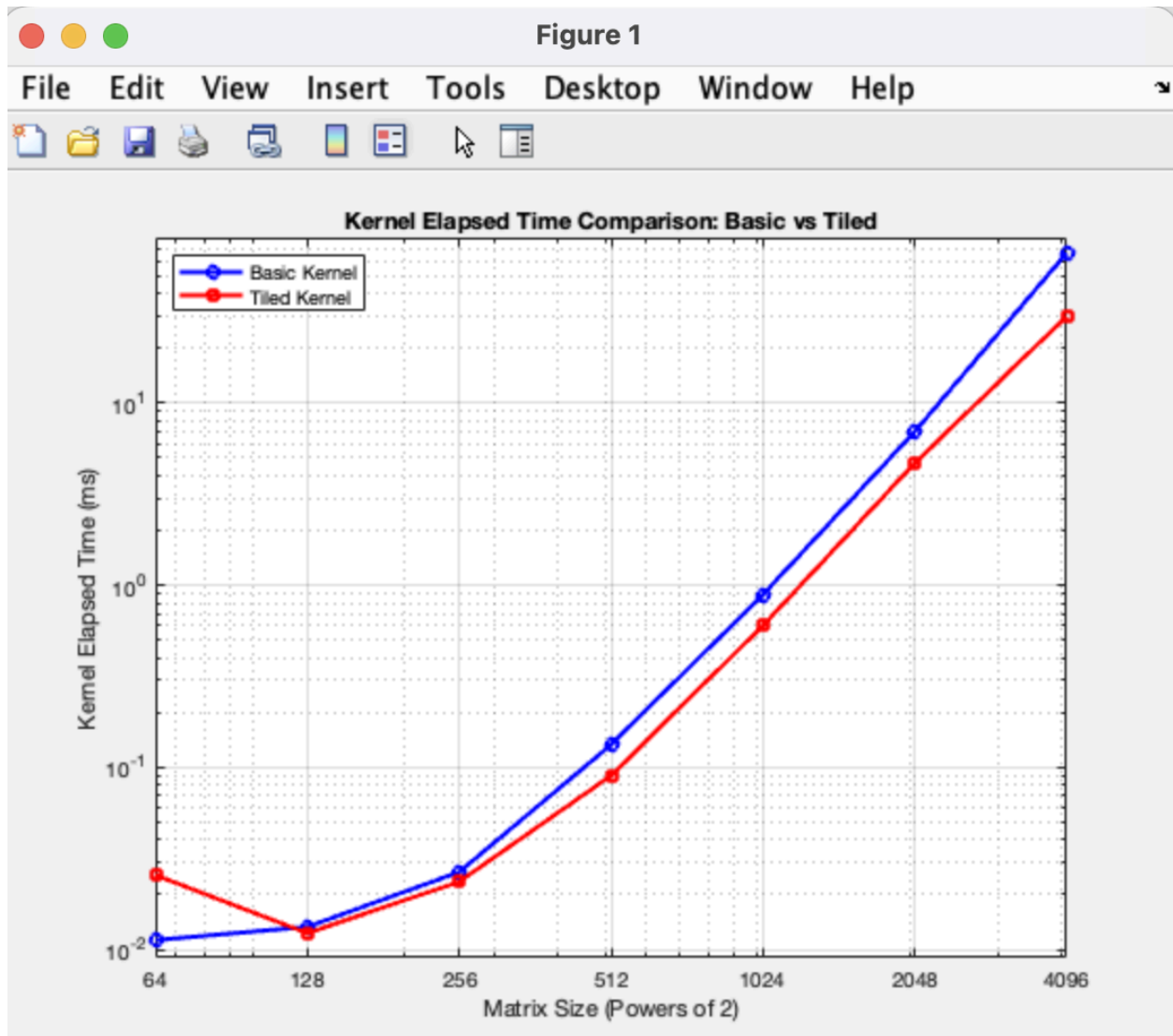


Brandon Nguyen, 827813045
COMPE596 - Spring 2026
2/27/2026

COMPE596 P05 - NVIDIA CUDA Tiled Dense Matrix Multiplication

1) Runtime Plot



2) Code Listing

// Main Program

```
#include <cuda_runtime.h>
#include <stdio.h>
#include <stdlib.h>
```

```
#define TILE_WIDTH 16
```

```
#define CUDA_CHECK(err) \
    if (err != cudaSuccess) { \
        printf("CUDA error: %s\n", cudaGetErrorString(err)); \
        exit(EXIT_FAILURE); \
    }
```

```
__global__ void matrixMultiply(float* A, float* B, float* C,
                               int m, int p, int n)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < m && col < n) {
        float sum = 0.0f;
        for (int k = 0; k < p; k++) {
            sum += A[row * p + k] * B[k * n + col];
        }
        C[row * n + col] = sum;
    }
}
```

```
__global__ void matrixTiledMultiply(float* A, float* B, float* C,
                                     int m, int p, int n)
{
    __shared__ float As[TILE_WIDTH][TILE_WIDTH];
    __shared__ float Bs[TILE_WIDTH][TILE_WIDTH];

    int row = blockIdx.y * TILE_WIDTH + threadIdx.y;
    int col = blockIdx.x * TILE_WIDTH + threadIdx.x;

    float sum = 0.0f;

    int numTiles = (p + TILE_WIDTH - 1) / TILE_WIDTH;

    for (int t = 0; t < numTiles; t++) {
```

```

    if (row < m && (t * TILE_WIDTH + threadIdx.x) < p)
        As[threadIdx.y][threadIdx.x] =
            A[row * p + t * TILE_WIDTH + threadIdx.x];
    else
        As[threadIdx.y][threadIdx.x] = 0.0f;

    if (col < n && (t * TILE_WIDTH + threadIdx.y) < p)
        Bs[threadIdx.y][threadIdx.x] =
            B[(t * TILE_WIDTH + threadIdx.y) * n + col];
    else
        Bs[threadIdx.y][threadIdx.x] = 0.0f;

    __syncthreads();

    for (int k = 0; k < TILE_WIDTH; k++) {
        sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
    }

    __syncthreads();
}

if (row < m && col < n)
    C[row * n + col] = sum;
}

void fillRandom(float* mat, int size) {
    for (int i = 0; i < size; i++) {
        mat[i] = (float)rand() / RAND_MAX;
    }
}

int main() {

    int sizes[] = {64, 128, 256, 512, 1024, 2048, 4096};
    int numSizes = 7;

    printf("Size Basic(ms) Tiled(ms)\n");

    for (int s = 0; s < numSizes; s++) {

```

```
int m = sizes[s];  
int p = sizes[s];  
int n = sizes[s];
```

```
float* hA = (float*)malloc(sizeof(float) * m * p);  
float* hB = (float*)malloc(sizeof(float) * p * n);  
float* hC = (float*)malloc(sizeof(float) * m * n);
```

```
fillRandom(hA, m * p);  
fillRandom(hB, p * n);
```

```
float *dA, *dB, *dC;  
CUDA_CHECK(cudaMalloc((void**)&dA, sizeof(float) * m * p));  
CUDA_CHECK(cudaMalloc((void**)&dB, sizeof(float) * p * n));  
CUDA_CHECK(cudaMalloc((void**)&dC, sizeof(float) * m * n));
```

```
CUDA_CHECK(cudaMemcpy(dA, hA, sizeof(float)*m*p,  
cudaMemcpyHostToDevice));  
CUDA_CHECK(cudaMemcpy(dB, hB, sizeof(float)*p*n,  
cudaMemcpyHostToDevice));
```

```
dim3 dimBlock(TILE_WIDTH, TILE_WIDTH);  
dim3 dimGrid((n + TILE_WIDTH - 1) / TILE_WIDTH,  
             (m + TILE_WIDTH - 1) / TILE_WIDTH);
```

```
matrixMultiply<<<dimGrid, dimBlock>>>(dA, dB, dC, m, p, n);  
cudaDeviceSynchronize();
```

```
cudaEvent_t start, stop;  
cudaEventCreate(&start);  
cudaEventCreate(&stop);
```

```
cudaEventRecord(start);  
matrixMultiply<<<dimGrid, dimBlock>>>(dA, dB, dC, m, p, n);
```

```

    cudaEventRecord(stop);

    cudaEventSynchronize(stop);

    float basicTime = 0.0f;
    cudaEventElapsedTime(&basicTime, start, stop);


    cudaEventRecord(start);
    matrixTiledMultiply<<<dimGrid, dimBlock>>>(dA, dB, dC, m, p, n);
    cudaEventRecord(stop);

    cudaEventSynchronize(stop);

    float tiledTime = 0.0f;
    cudaEventElapsedTime(&tiledTime, start, stop);

    printf("%d %f %f\n", m, basicTime, tiledTime);

    free(hA);
    free(hB);
    free(hC);

    cudaFree(dA);
    cudaFree(dB);
    cudaFree(dC);

    cudaEventDestroy(start);
    cudaEventDestroy(stop);
}

return 0;
}

```

// Makefile

```

NVCC = nvcc
TARGET = P05
SRC = P05.cu

```

```

all: $(TARGET)

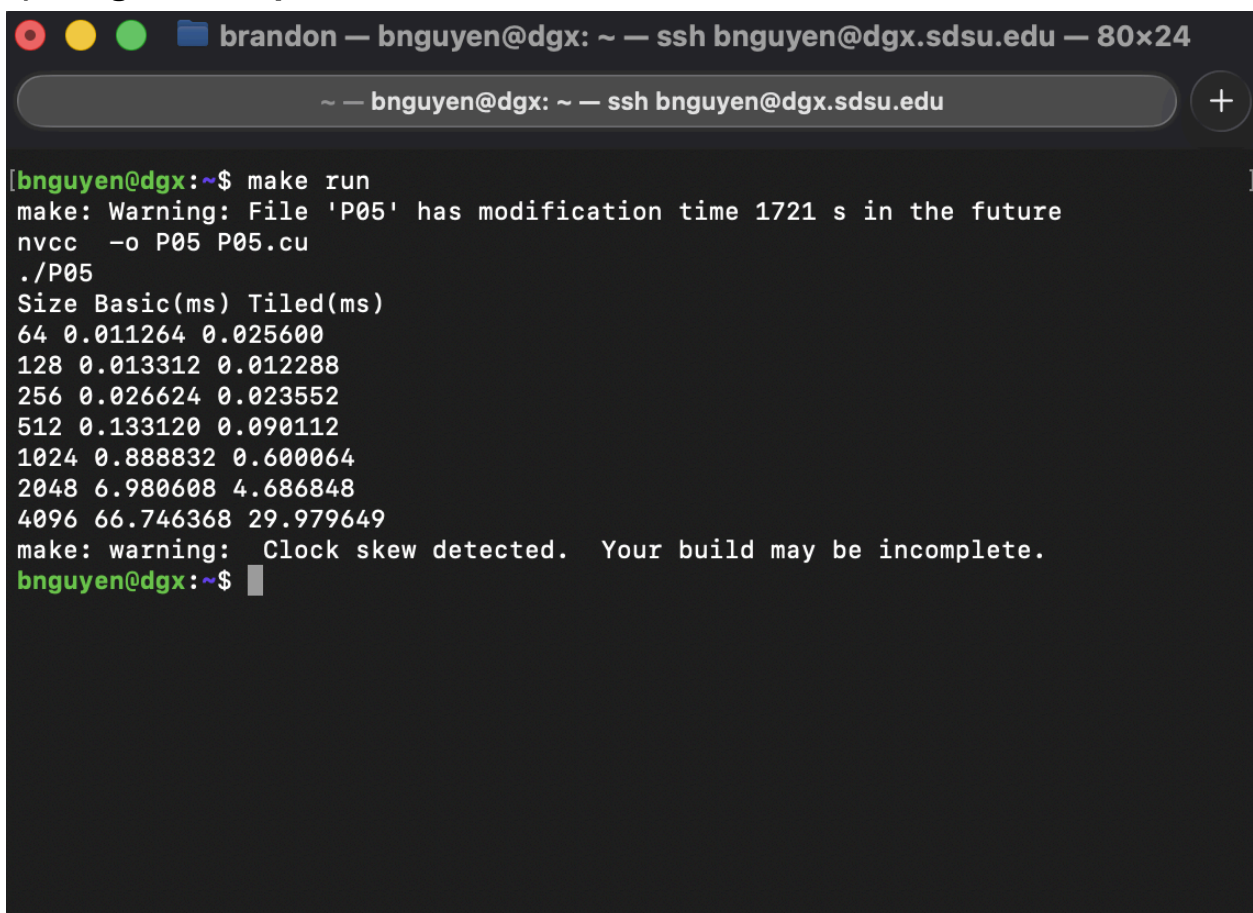
```

```
$(TARGET): $(SRC)
    $(NVCC) $(CFLAGS) -o $(TARGET) $(SRC)
```

```
run: $(TARGET)
    .$(TARGET)
```

```
clean:
    rm -f $(TARGET)
```

3) Program Output



```
[bnguyen@dgx:~$ make run
make: Warning: File 'P05' has modification time 1721 s in the future
nvcc -o P05 P05.cu
./P05
Size Basic(ms) Tiled(ms)
64 0.011264 0.025600
128 0.013312 0.012288
256 0.026624 0.023552
512 0.133120 0.090112
1024 0.888832 0.600064
2048 6.980608 4.686848
4096 66.746368 29.979649
make: warning: Clock skew detected. Your build may be incomplete.
bnguyen@dgx:~$
```

4) MATLAB Plot Script

```
sizes = [64, 128, 256, 512, 1024, 2048, 4096];
basic_time = [0.011264, 0.013312, 0.026624, 0.133120, 0.888832, 6.980608,
66.746368];
tiled_time = [0.025600, 0.012288, 0.023552, 0.090112, 0.600064, 4.686848,
29.979649];
```

```

figure;
plot(sizes, basic_time, '-o', 'Color', 'b', 'LineWidth', 2, 'MarkerSize', 6); hold on;
plot(sizes, tiled_time, '-s', 'Color', 'r', 'LineWidth', 2, 'MarkerSize', 6);

set(gca, 'XScale', 'log', 'YScale', 'log');

xticks(sizes);
xticklabels(arrayfun(@num2str, sizes, 'UniformOutput', false));

xlabel('Matrix Size (Powers of 2)');
ylabel('Kernel Elapsed Time (ms)');
title('Kernel Elapsed Time Comparison: Basic vs Tiled');
legend('Basic Kernel', 'Tiled Kernel', 'Location', 'northwest');
grid on;

xlim([min(sizes) max(sizes)]);
ylim([min([basic_time, tiled_time])*0.8 max([basic_time, tiled_time])*1.2]);

```

5) Discussion

In P05, we are tasked with writing a program using the DGX server to implement a basic dense matrix multiplication CUDA kernel and a tiled dense matrix multiplication CUDA kernel that uses shared memory in order to ultimately compare their runtimes. Once again, we will be plotting the runtime results between the two to visually see the crossover that happens and will also be using a Makefile to efficiently run our program. According to the output terminal and plotted graph, I used powers of two ranging from 2^6 to 2^{12} otherwise known as 64 to 4096, totaling seven test values. Here we learn that when the matrix dimension is 64, the basic method is faster than the tiled method. Then, at size 128 is the expected crossover region where the tiled method will either even up or surpass the basic method, and size 256 is when the tiled method fully surpasses in runtime speed. As the matrix dimensions increase beyond size 256, the tiled method runtime just becomes even faster due to shared memory access. Therefore, the conclusion that we can form for our understanding is that as the matrix dimension size increases, the tiled method will become faster and faster because it has shared memory access capabilities to load and reuse data while the basic method does not. With that being said, the fifth programming assignment was good in helping us understand the correlation between shared memory access and matrix multiplication in CUDA.